LAU
Lebanese American University
Chartered in the State of New York
www.lau.edu.lb

SOE
School of Engineering
Department of Electrical & Computer Engineering
www.lau.edu.lb/ece

BYBLOS CAMPUS	BEIRUT CAMPUS	NEW YORK HEADQUARTERS
P.O. Box: 36 Byblos, Lebanon Tel: +961 9 547 262 Fax: +961 9 546 262	P.O. Box: 13-5053 Chouran Beirut 1102 2801, Lebanon Tel: + 961 1 786 456 Fax: +961 1 867 098	211 E. 46th Street New York, NY 10017-2935, USA Tel: +1 212 203 4333 Fax: +1 212 784 6597

Intelligent Data Processing and Applications

COE 543/743

Chapter 6 - Structural Similarity Approximation Methods

Joe M. TEKLI, PhD
Computer Software Engineering

Ch. 6. Structural Similarity Approximation

Chapter Contents

1. Introduction
2. Prerequisites
   - Set-based Similarity
   - Vector-based Similarity
3. Semi-structured Data Similarity Methods
   - Tag-based
   - Edge-based
   - Path-based
   - Fast Fourier Transform

Ch. 6. Structural Similarity Approximation

1. Introduction

- In certain similarity-based applications, users might want to **speed-up** the process
  - Typical in **data search** and **querying** applications

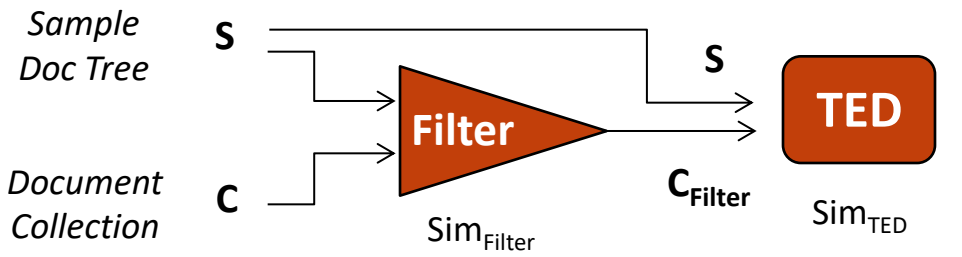
- **Filter-Refinement** architecture in **query processing**:
  1. Run a filter step, using a fast approximation, e.g., $\mathbf{Sim_{Filter}}$, of the main similarity measure, e.g., $\mathbf{Sim_{TED}}$
     - To compare a sample (query) document to those kept in the collection
  2. Only those document trees in the data collection which are most similar (following $\mathbf{Sim_{Filter}}$) to the sample (query) document are run through the main measure $\mathbf{Sim_{TED}}$

↳ Reducing query processing time

Ch. 6. Structural Similarity Approximation

1. Introduction

- **Filter-Refinement** architecture in **query processing**:



$Ans(S, Thresh_{Sim}) = \langle D_1, \ldots, D_n \rangle$

*The query answer is a sequence of documents D_i most similar to **S** following a certain similarity threshold $Thresh_{Sim}$ ranked using similarity $Sim_{TED}(S, D_i)$*

- Formally:

Selection

$$Filter = \{ D_i \in C \mid Sim_{Filter}(S, D_i) \geq Thresh_{Sim} \wedge \forall D_j \in C, Sim_{Filter}(S, D_i) \geq Sim_{Filter}(S, D_j) \}$$

Ordering

$\Rightarrow Sim_{TED}(S, D_i)$

Ch. 6. Structural Similarity Approximation

1. Introduction

- **Filter-Refinement** can be used with most similarity-based applications, e.g.:
  - **Clustering**:
    1. Apply clustering using approximation similarity method
       - Producing an initial set of clusters
    2. Apply the main (TED-based) similarity measure on each cluster of the initial set
  - **Data integration**:
    1. Given two data collections C_1 and C_2 to integrate, compute pair-wise similarity between documents D_i^1 and D_j^2 from C_1 and C_2 using approximate measure
       - Disregarding non-similar documents and producing filtered collections C_1' and C_2'
    2. Apply the main (TED-based) similarity measure on pair-wise documents of the filtered collections to identify most similar ones for merging

Ch. 6. Structural Similarity Approximation

1. Introduction

- **Filter properties**:
  - Considerably **easier to process** than the main (TED-based) function
  - **Filters out** a substantial part of the documents
  - Verifies the ***completeness*** property w.r.t. the main (TED-based) function
    - Filter must not allow any **false drop-outs**

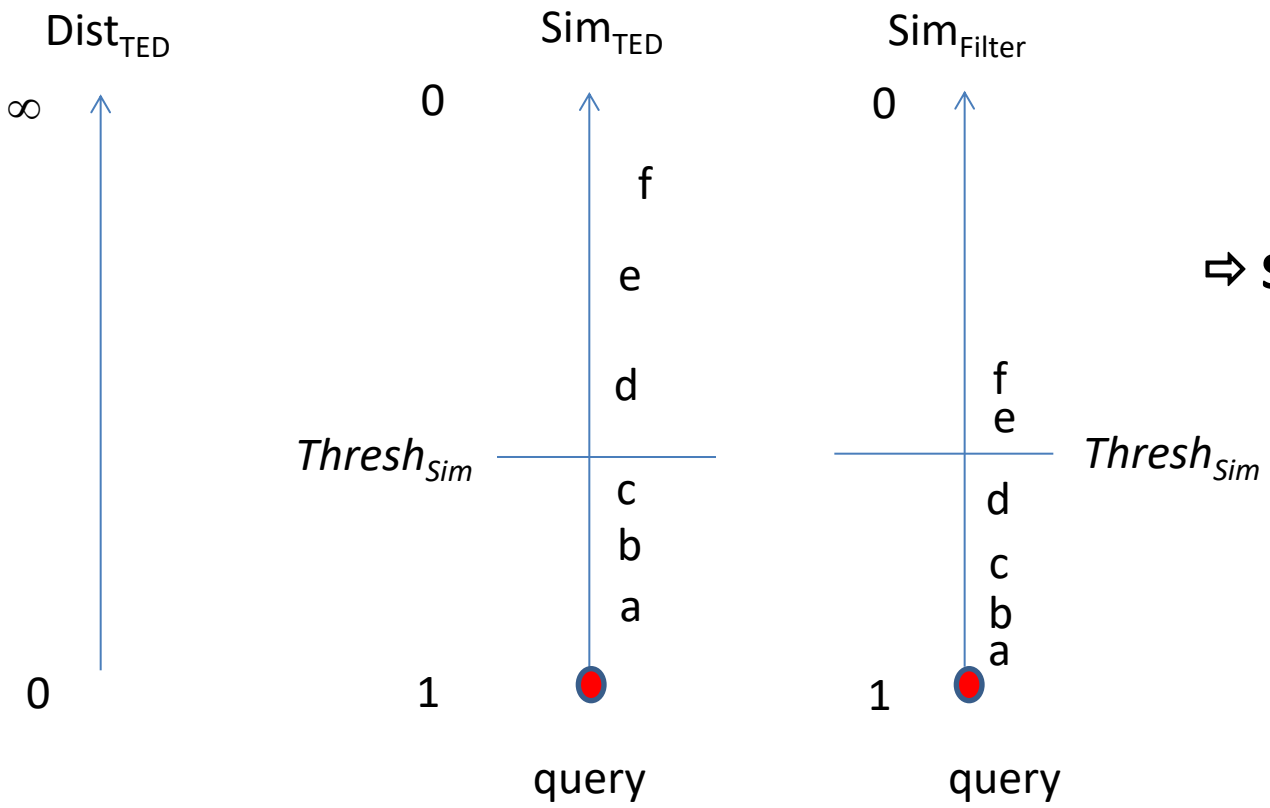
⇨ All document trees in the collection, $\forall$ $\mathbf{D_i} \in \mathbf{C}$, which are deemed similar to the sample document **S** w.r.t. the main similarity measure $\mathbf{Sim_{TED}}$, should be included in the filtered set, $\mathbf{D_i} \in \mathbf{C_{Filter}}$

↳ **Upper bound** of TED

⇨ $Sim_{Filter} \geq Sim_{TED}$

Ch. 6. Structural Similarity Approximation

1. Introduction



⇨ $\mathbf{Sim_{Filter} \geq Sim_{TED}}$

Ch. 6. Structural Similarity Approximation

1. Introduction

- More formally:
  - **Filter *Completeness*:** Given a similarity measure $\mathbf{Sim_{TED}}$ and a filter characterized by similarity measure $\mathbf{Sim_{Filter}}$, the filter is said to be complete w.r.t. $\mathbf{Sim_{TED}}$ if $\mathbf{Sim_{Filter}}$ is an **upper bound** of $\mathbf{Sim_{TED}}$
  - **Upper Bound Function:** Let **C** be a collection of documents, a similarity function **Sim'** is an upper bound of function **Sim**, if: $\forall\ \mathbf{D_i, D_j \in C}$, $\mathbf{Sim'(D_i, D_j) \geq Sim\ (D_i, D_j)}$

- With an **upper bound** similarity measure, it is possible to safely filter out all documents trees that have a filter similarity $\mathbf{Sim_{Filter}}$ less than the minimum acceptable similarity degree, i.e., eliminating all $\mathbf{D_i}$ / $\mathbf{Sim_{Filter}}(S, D_i) < \mathbf{Thresh_{Sim}}$

  ⇨ Eliminating candidate document trees which are outside the minimum relevant (user-chosen) similarity range

Ch. 6. Structural Similarity Approximation

1. Introduction

- Approximate similarity functions could also be useful in producing applications that run faster without taking the time to do accurate similarity computations
  - **Without** any **refinement** step
    - For instance, designing faster clustering, classification, or querying approaches
      - Which might not be very accurate
      - But which can get the job done in lesser time
        - With acceptable quality levels

↳ Need for structural similarity **approximation** measures

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.1. Set-based Similarity

Various measures exist for evaluating **similarity** between **sets** of elements

1. Intersection: Given two sets A and B

$$\mathbf{Sim_{Intersection}(A, B) = |A \cap B|} \quad \in [0, \min(|A|, |B|)]$$

- Minimum similarity = 0 and maximum similarity = min(|A|,|B|)

↳ Not normalized

- Since values are not comprised in the [0, 1] interval

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.1. Set-based Similarity

Various measures exist for evaluating **similarity** between **sets** of elements

2. <u>Jaccard Coefficient</u>: Given two sets A and B

$$\mathbf{Sim}_{\mathbf{Jaccard}} = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \in [0,1]$$

- **Normalized measure**
- **More accurate** than intersection similarity since it considers both:
  - The commonality between the sets
  - The differences between the sets

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.1. Set-based Similarity

Various measures exist for evaluating **similarity** between **sets** of elements

3. <u>Dice Coefficient</u>: Given two sets A and B

$$\mathbf{Sim_{Dice}} = \frac{2\times|A\cap B|}{|A|+|B|} \quad \in [0,1]$$

- **Normalized measure**
- **Alternative to Jaccard** used in certain applications

↳ Dice distance does not verify **Triangular Inequality**

- Qualifies as a **semi-metric**

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.2. Multi-Set based Similarity

- A **multi-set** is a generalization of the concept of a set
  - Unlike a set, it allows multiple instances of the same element
    - **Multiplicity** of an element: number of occurrences of the element in the multi-set

- For example:
  - {*a*, *b*, *c*} contains only elements *a*, *b*, and *c* each having multiplicity 1
    - ⇨ Traditional set, also called: **unique** set
  - {*a*, *a*, *b*, *c*, *c*, *c*}, is a **multi-set** where *a*, *b*, and *c* have multiplicities 2, 1, and 3 respectively
    - It is usually represented as a unique set of doublets: **(*e*, *mult*(*e*))**
      - Where ***e*** designates an element and ***mult*(*e*)** its multiplicity value
        - ⇨ {*a*, *a*, *b*, *c*, *c*, *c*} ≡ {(a, 2), (b, 1), (c, 3)}

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.2. Multi-Set based Similarity

Measures for comparing unique sets can be extended for comparing **multi-sets**

1. Intersection: Given two multi-sets A and B

$$\mathbf{Sim_{Intersection}(A, B) = |A \cap B|} = \sum_{i \,/\, A_i = B_i} \boldsymbol{Min(mult(A_i), mult(B_i))} \in [0, \min(\sum_i mult(A_i), \sum_i mult(B_i))]$$

- Minimum similarity = 0 and maximum similarity = $\min(\sum_i mult(A_i), \sum_i mult(B_i))$

↳ Not normalized

- Since values are not comprised in the [0, 1] interval

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.2. Multi-Set based Similarity

Measures for comparing unique sets can be extended for comparing **multi-sets**

2. Jaccard Coefficient: Given two multi-sets A and B

$$\mathbf{Sim}_{\mathbf{Jaccard}} = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \in [0,1]$$

$$= \frac{\sum_{i \,/\, A_i = B_i} Min(mult(A_i), mult(B_i))}{\sum_i mult(A_i) + \sum_i mult(B_i) - \sum_{i/\, A_i = B_i} Min(mult(A_i), mult(B_i))}$$

- **Normalized measure**
- **More accurate** than intersection similarity since it considers both:
  - The commonality and the differences between the multi-sets

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.2. Multi-Set based Similarity

Measures for comparing unique sets can be extended for comparing **multi-sets**

3. <u>Dice Coefficient</u>: Given two sets A and B

$$Sim_{Dice} = \frac{2 \times |A \cap B|}{|A| + |B|} = \frac{2 \times \sum_{i \,/\, A_i = B_i} Min(mult(A_i), mult(B_i))}{\sum_i mult(A_i) + \sum_i mult(B_i)} \in [0,1]$$

- **Normalized measure**
- **Alternative to Jaccard** used in certain applications

↳ Dice distance does not verify **Triangular Inequality**

- Qualifies as a **semi-metric**

Ch. 6. Structural Similarity Approximation

2. Prerequisites

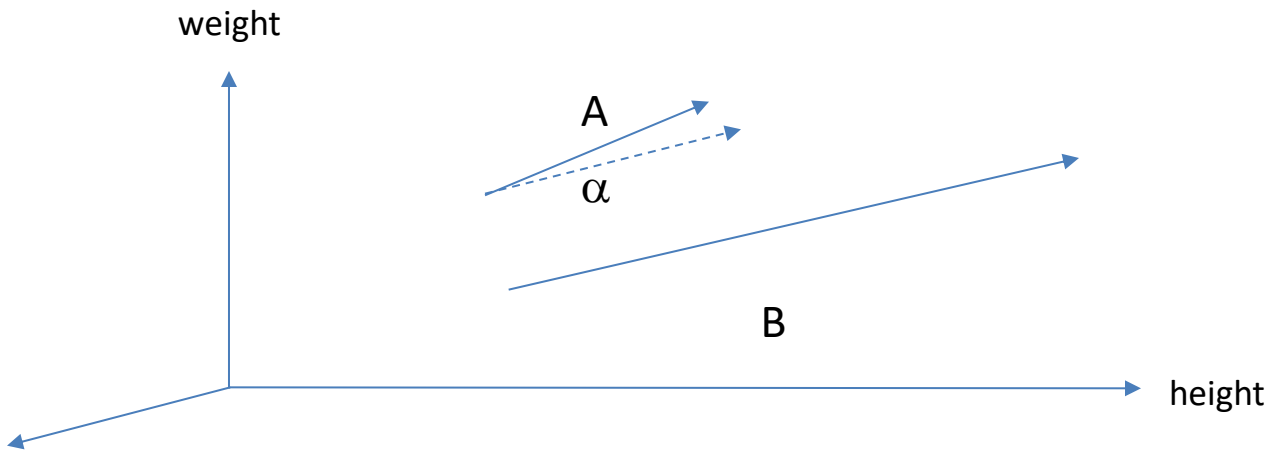
2.3. Vector-based Similarity

- Various measures exist for evaluating **similarity** between **vectors** in an *N*-dimensional vector space
  - We can organize them in two main groups
    - **Correlation** measures, evaluating **dependence** between vector shapes, such as:
      - Cosine
      - Pearson Correlation Confident (PCC)
    - **Distance** measures, evaluating **separation** between vectors, such as:
      - Euclidian distance
      - Manhattan distance

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity



$Human_{Joe}$= { (Height, 80), (Weight, 180), …, …, .., … }

$Human_{Mohamad}$= {(Height, 90), (Weight, 170) , …., …., …, …, }

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Correlation Measures:

1. <u>Cosine measure</u>: Given two vectors $\vec{A}$ and $\vec{B}$ in an *N*-dimensional space

$$\mathbf{Sim}_{\mathbf{Cosine}}(\vec{A},\vec{B}) = \frac{\sum_{i=1\ldots N} \mathbf{A}_i \times \mathbf{B}_i}{\sqrt{\sum_{i=1\ldots N} \mathbf{A}_i^2 \times \sum_{i=1\ldots N} \mathbf{B}_i^2}} \quad \in [-1,1]$$

- Computed based on the **scalar product**: $\vec{A} \,.\, \vec{B} = |A| \times |B| \times cos(\alpha)$
  - ⇨ Takes into account the **angle between** two **vectors**, **not their modules**
- Measure produces values comprised in [-1, 1] instead of [0, 1]
  - Linear correlation measure: 1 represents maximum correlation
    0 represents no correlation
    -1 represents negative correlation

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Correlation Measures:

2. <u>Pearson Correlation Coeficient</u>: Given two vectors $\vec{A}$ and $\vec{B}$

$$\mathbf{Sim_{PCC}}(\vec{A},\vec{B}) = \frac{\sum_{i=1\ldots N} (A_i - \bar{A}) \times (B_i - \bar{B})}{\sqrt{\sum_{i=1\ldots N} (A_i - \bar{A})^2 \times \sum_{i=1\ldots N} (B_i - \bar{B})^2}} \quad \in [-1, 1]$$

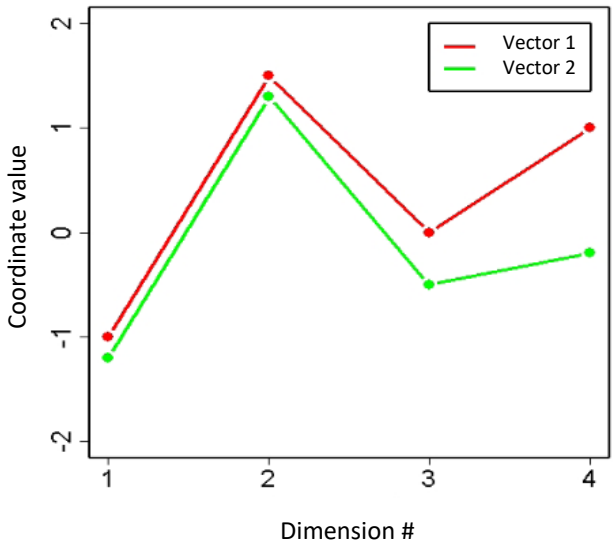
- $\bar{A}$ and $\bar{B}$ represent the average of the values of vectors $\vec{A}$ and $\vec{B}$ respectively
  - Comparable to cosine except that individual **vector coordinates** are **centered**
- Measure produces values comprised in [-1, 1] instead of [0, 1]
  - Linear correlation measure: 1 represents maximum correlation
    0 represents no correlation
    -1 represents negative correlation

Ch. 6. Structural Similarity Approximation

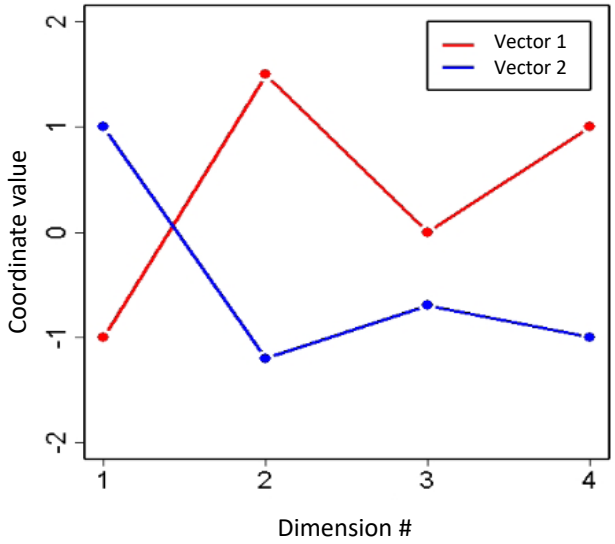
2. Prerequisites

2.3. Vector-based Similarity

Correlation Measures:



Positive correlation



Negative correlation

We can use **absolute correlation** to capture both positive and negative correlation

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Distance Measures:

3. <u>Euclidian Distance</u>: Given two vectors $\vec{A}$ and $\vec{B}$ in *N*-dimensional space

$$\mathbf{Sim_{Euclid}}(\vec{A},\vec{B}) = \frac{1}{1+\mathbf{Dist_{Euclid}}(\vec{A},\vec{B})} \quad \in \ ]0,1]$$

$$\Rightarrow \text{having} \quad \mathbf{Dist_{Euclid}}(\vec{A},\vec{B}) = \sqrt{\sum_{i=1\ldots N} (A_i - B_i)^2} \quad \in [0,+\infty[$$

- Considers the **distance** between two vector, taking into account their **modules**

]

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Distance Measures:

4. <u>Manhattan Distance</u>: Given two vectors $\vec{A}$ and $\vec{B}$ in *N*-dimensional space

$$\mathbf{Sim_{Manh}}(\vec{A},\vec{B}) = \frac{1}{1+\mathbf{Dist_{Manh}}(\vec{A},\vec{B})} \quad \in \ ]0,1]$$

$$\Rightarrow \text{having} \quad \mathbf{Dist_{Manh}}(\vec{A},\vec{B}) = \sum_{i=1\ldots N} |\mathbf{A_i} - \mathbf{B_i}| \quad \in [0,+\infty[$$

- **Taxicab geometry** in which the distance between two points is the sum of the absolute differences of their Cartesian coordinate
  - ⇨ Lower bound for *Euclidian distance* and for typical *edit distance* functions

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Distance Measures:

5. <u>Tanimoto Distance</u>: Given two vectors $\vec{A}$ and $\vec{B}$ in *N*-dimensional space

$$\mathbf{Sim_{Tanimoto}}(\vec{A},\vec{B}) = \frac{\vec{A}\times\vec{B}}{\left|\vec{A}\right|^2 + \left|\vec{B}\right|^2 - \vec{A}\times\vec{B}} \quad \in [0,1]$$

$$\Rightarrow \text{having} \quad \mathbf{Dist_{Tanimoto}}(\vec{A},\vec{B}) = 1 - \mathbf{Sim_{Tanimoto}}(\vec{A},\vec{B}) \quad \in [0,1]$$

- When applied on positive weighted vectors, the Tanimoto distance comes down to **Jaccard distance**, measuring the dissimilarity between vectors

Ch. 6. Structural Similarity Approximation

2. Prerequisites

2.3. Vector-based Similarity

Distance Measures:

5. <u>Dice Distance</u>: Given two vectors $\vec{A}$ and $\vec{B}$ in *N*-dimensional space

$$\mathbf{Sim}_{\mathbf{Dice}}(\vec{\mathbf{A}},\vec{\mathbf{B}}) = \frac{2\times\vec{\mathbf{A}}\times\vec{\mathbf{B}}}{\left|\vec{\mathbf{A}}\right|^2+\left|\vec{\mathbf{B}}\right|^2} \quad \in [0,1]$$

$$\Rightarrow \text{having} \quad \mathbf{Dist}_{\mathbf{Dice}}(\vec{\mathbf{A}},\vec{\mathbf{B}}) = 1-\mathbf{Sim}_{\mathbf{Dice}}(\vec{\mathbf{A}},\vec{\mathbf{B}}) \quad \in [0,1]$$

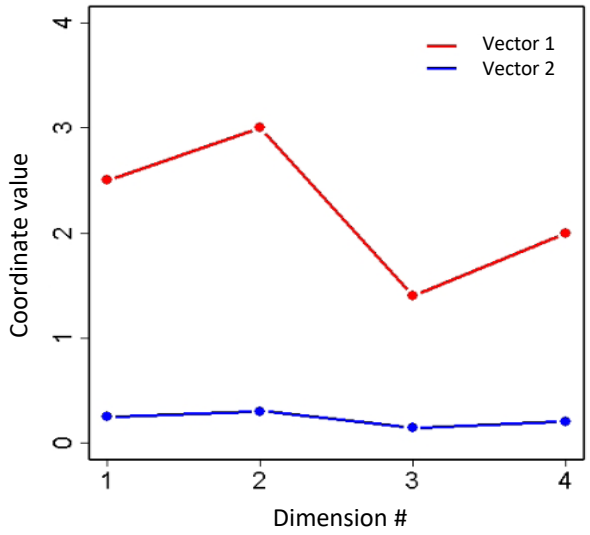
- While comparable to Tanimoto (Jaccard) distance, yet the Dice distance does not qualify as a metric since it does not verify the ***Triangular Inequality***
  - It is a **semi-metric**

Ch. 6. Structural Similarity Approximation

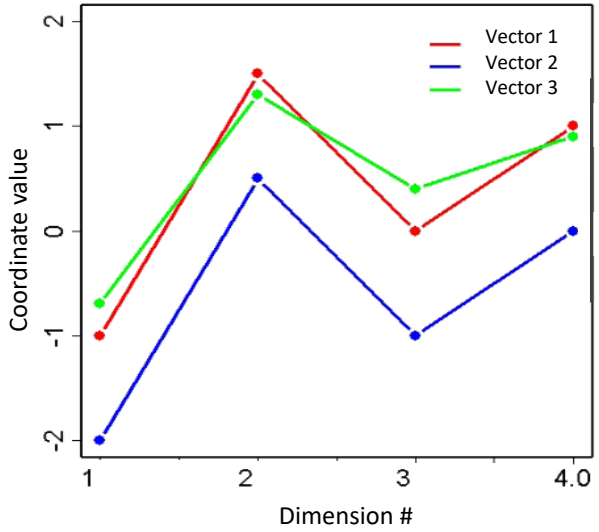
2. Prerequisites

2.3. Vector-based Similarity

Distance Measures:



$\mathbf{Sim_{Cosine}(V1, V2) = 1}$,
Yet $\mathbf{Sim_{Euclid}(V1, V2)}$ is much less



$\mathbf{Sim_{Cosine}(V1, V2) = 1 > Sim_{Cosine}(V1, V3)}$
Yet $\mathbf{Sim_{Euclid}(V1, V2) < Sim_{Euclid}(V1, V3)}$

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

- Transforming a semi-structured document tree into a **tag**-based representation [1, 2]
  - A tag usually represents an element/attribute name (label)
    - A tag can also represent an element/attribute value when content is considered
      - Content can be considered in one single element (as one tag)
      - Content can be tokenized into multiple elements (multiple tags)
        - Refer to document tree representation in Ch. 2

- Different tag-based representations can be used:
  - Set-based
  - Multi-set based
  - Vector-based

Exploiting corresponding similarity measures

[1] L. Candillier, I. Tellier and F. Torre. Transforming XML Trees for Efficient Classification and Clustering. *Proceedings of the Workshop of the Initiative for the Evaluation of XML Retrieval (INEX'05), (2005) pp. 469-480*

[2] L. Candillier, I. Tellier. F. Torre and O. Bouquet. SSC: Statistical Clustering. In *Perner P. and Imiya A. eds.: 4th Inter. Conf. on Machine Learning and Data Mining in Pattern Recognition,* (2005) pp. 100-109.

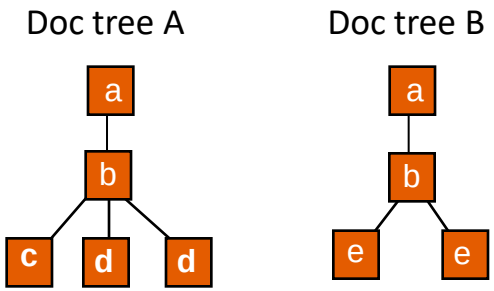
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

3.1.1. Set-based Approach

- Consider document trees A and B
  - Set-based representations of A and B:
    - A_{set} = {'a', 'b', 'c', 'd'}
    - B_{set} = {'a', 'b', 'e'}



↳ Compared using any set-based similarity measure

⇨ Same tag occurrences are **not** taken into account!

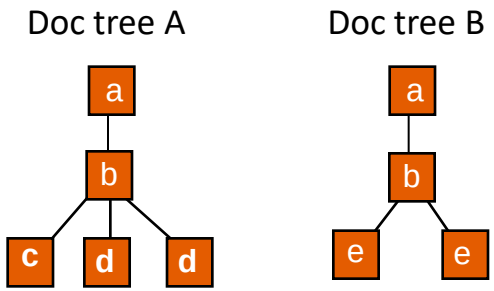
- Complexity: average linear, i.e., **O(*k* × *N*)** where *N* = number of distinct tags labels in A and B, and *k* is a variable factor usually equivalent to a small multiple of *N*

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

3.1.2. Multi-set Approach



- Consider document trees A and B
  - Multi-set based representations of A and B:
    - $A_{multi\text{-}set}$ = {('a', 1), ('b', 1), ('c', 1), ('d', 2)}
    - $B_{multi\text{-}set}$ = {('a', 1), ('b', 1), ('e', 2)}

↳ Compared using any multi-set similarity measure

⇨ Taking into account the number of tag occurrences (multiplicity)

- More accurate than set-based approach

- Complexity: **O(*k* × *N*)** where *N* = number of distinct tag labels in A and B, and *k* is a variable factor

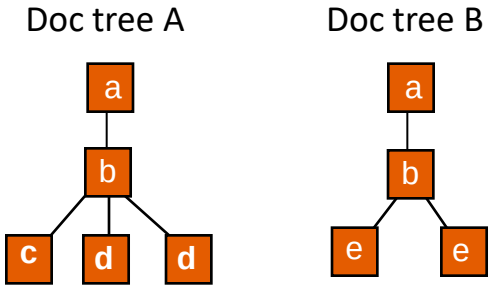
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

3.1.3. Vector-based Approach

- Consider document trees A and B



  - Vector-based representations of A and B:

	‘a’	‘b’	‘c’	‘d’		‘a’	‘b’	‘é’
A_{vector} =	(1 ,	1,	1,	2)	B_{vector} =	(1 ,	1,	2)

- Vector dimensions represent distinct tag labels
- Vector weights represent tag occurrence frequencies

↳ Compared using any vector-based similarity measure

- Complexity: **O(*k* × *N*)** where *N* = number of distinct tag labels in A and B and *k* is a variable

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

3.1.3. Vector-based Approach

- NOTE: When comparing two vectors of different dimensions
  - They have to be represented in a common (same) dimensional space
    - In order to apply vector-based similarity measures
      - Intuitive approach: **combining dimensions**

	‘a’	‘b’	‘c’	‘d’	‘e’
A_{vector} =	(1 ,	1,	1,	2,	0)
B_{vector} =	(1 ,	1,	0,	0,	2)

↳ Simple, yet the number of dimensions can uselessly explode

⇨ Also known as **dimensionality problem**

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.1. Tag-based Similarity

3.1.3. Vector-based Approach

- More sophisticated approaches to handle the dimensionality problem:
  - **Dimension selection**
    - Classification (supervised learning) methods can be used to select the set of dimensions which are most useful in describing the document trees at hand
      - E.g., *Correlation-based Feature Selection* (CFS) which studies correlations between dimensions to disregard redundant (useless) ones
  - **Dimension reduction**
    - Transforming original dimensions into a new set of reduced dimensions which are (hypothetically) more descriptive of original ones
      - E.g., *Principal Component Analysis* (PCA) producing new dimensions ranked based on their descriptive power

        ⇨ Latent semantic processing (unsupervised learning) techniques

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.2. Edge-based Similarity

- Transforming a semi-structured document tree into an **edge**-based representation [1]
  - An edge is an ordered doublet of tree nodes **(n_1, n_2)**, represented as **n_1/n_2** where:
    - **n_1** is the parent node of **n_2** in the document tree
      - Nodes have as labels corresponding node tag names

- Different tag-based representations can be used:
  - Set-based
  - Multi-set based
  - Vector-based

Exploiting corresponding similarity measures

[1] H.P. Kriegel and S. Schönauer. Similarity Search in Structured Data. In *Proceedings of the 5th International Conference on Data Warehousing and Knowledge Discovery (DaWaK 03), Czech Republic, (2003) pp. 309-319.*

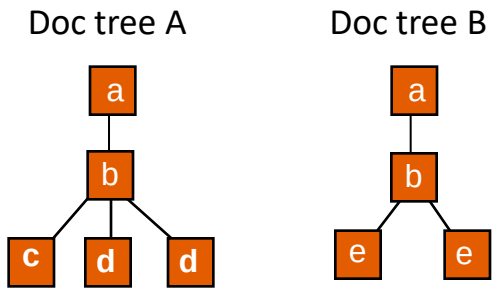
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.2. Edge-based Similarity

3.2.1. Set-based Approach

- Consider document trees A and B
  - Set-based representations of A and B:
    - A_{set} = {'a'/'b', 'b'/'c', 'b'/'d'}
    - B_{set} = {'a'/'b', 'b'/'e'}



Compared using any set-based similarity measure

⇨ Same edge occurrences are **not** taken into account!

- Complexity: average linear, i.e., **O(*k* × *N*)** where *N* = number of distinct edges in A and B, and *k* is a variable factor

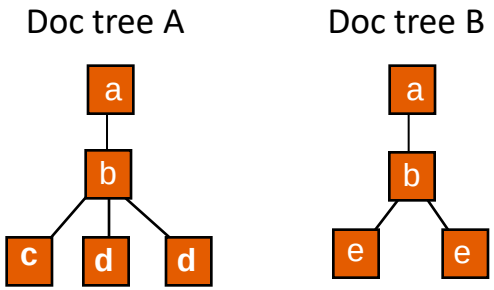
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.2. Edge-based Similarity

3.2.2. Multi-set Approach

- Consider document trees A and B
  - Multi-set-based representations of A and B:
    - A_{set} = {('a'/'b', 1), ('b'/'c', 1), ('b'/'d', 2)}
    - B_{set} = {('a'/'b', 1), ('b'/'e', 2)}



Compared using any multi-set similarity measure

⇨ Taking into account the number of edge occurrences (multiplicity)

- More accurate than set-based approach

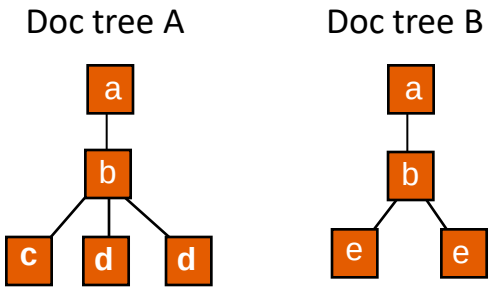
- Complexity: **O(*k* × *N*)** where *N* = number of distinct edges in A and B, and *k* is a variable

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.2. Edge-based Similarity

3.2.3. Vector-based Approach



- Consider document trees A and B
  - Vector-based representations of A and B:

	‘a’/’b’	‘b’/’c’	‘b’/’d’		‘a’/’b’	‘b’/’e’
A_{vector} = (	1 ,	1,	2)	B_{vector} = (	1 ,	2)

- Vector dimensions represent distinct tree edges
- Vector weights represent edge occurrence frequencies

↳ Compared using any vector-based similarity measure

- Complexity: **O(*k* × *N*)** where *N* = number of distinct edges in A and B and *k* is a variable

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.2. Edge-based Similarity

- NOTE:
  - Edge-based approaches detect more structural clues than tag-based methods
    - Taking into account parent/child structural relations

      ↳ Increasing structural **precision**

  - Yet they might miss certain tag-based similarities
    - Even if these are not structurally coherent, yet they might be deemed valuable in certain applications…

      ↳ Decreasing overall **recall**

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

- Transforming a semi-structured document tree into a **path**-based representation [1, 2]
  - An path is an sequence of tree nodes $<n_1, n_2, \ldots, n_p>$, noted $n_1/ n_2 /\ldots/n_p$ where
    - n_i is the parent node of n_{i+1} in the document tree
      - Nodes have as labels corresponding node tag names

- Different variations of path-based representations exit:
  - ***Root Paths***, ***All Paths***, and ***XPaths***
    - Each can be represented following different mathematical models:
      - Set-based
      - Multi-set based
      - Vector-based

Exploiting corresponding similarity measures

[1] D. Buttler. A Short Survey of Document Structure Similarity Algorithms. In *Proceedings of the 5th International Conference on internet Computing, USA, (2004) pp. 3-9.*

[2] D. Rafiei, D. Moise and D. Sun. Finding Syntactic Similarities between XML Documents. In *Proceedings of the 17th International Conference on Database and Expert Systems Applications (DEXA), (2006) pp. 512-516.*

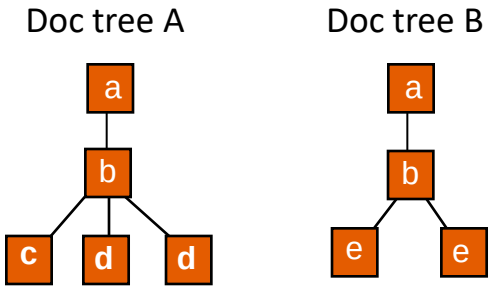
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.1. Root Path Approach

- **Root path:** a path starting at the root and ending at the a leaf in the document tree



- Consider document trees A and B
  - Considering the vector-based model

A_{vector} = (1 , 2) with ‘a’/’b’/’c’ and ‘a’/’b’/’d’

B_{vector} = (2) with ‘a’/’b’/’e’

↳ Compared using any vector-based similarity measure

- Complexity: **O(*k* × *N*)** where *N* = number of distinct root paths in A and B, and *k* is a variable factor

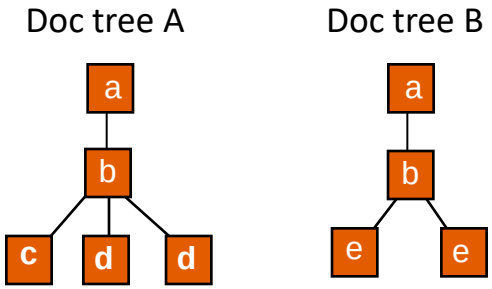
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.2. All Path Approach

- Considering all paths in the document tree
- Consider document trees A and B
  - Considering the vector-based model



	‘a’	‘b’	‘c’	‘d’	‘a’/’b’	‘b’/’c’	‘b‘/’d’	a’/’b’/’c’	‘a’/’b’/’d’
A_{vector} = (	1 ,	1	1	2	1	1	2	1	2)

	‘a’	‘b’	‘e’	‘a’/’b’	‘b’/’e’	‘a’/’b’/’e’
B_{vector} = (	1	1	2	1	2	2)

Compared using any vector-based similarity measure

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.2. All Path Approach

- Considering all paths in the document tree
- Complexity: **O(*k* × *N*)** where *N* = number of distinct paths in A and B, and *k* is a variable
  - Comes down to **O(*k* × *N* × l²)** where:
    - N is the number of distinct root paths
    - l is the maximum length of a root path
    - *k* is a variable factor

↳ Close to TED-based polynomial complexity

- Yet less precise in detecting structural clues

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.3. XPath Approach

- Transforming the semi-structured document tree into an XPath-based representation [1]
- Classic paths underline **parent/child** relationships in the document tree
  - Ignoring **sibling** information
- Yet, XPaths incorporate some sibling information
  - Underlining, for a given node, how many preceding siblings have the same label
    - It does not capture sibling information about nodes whose labels are different from the given node

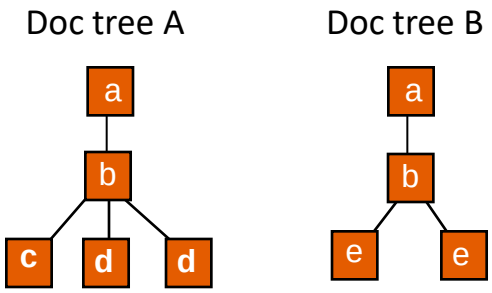
[1] S. Joshi, N. Agrawal, R. Krishnapuram and S. Negi. A Bag of Paths Model for Measuring Structural Similarity in Web Documents. *ACM SIGKKD Conference on Knowledge Discovery and Data Mining,* USA, (2003), pp. 577-582.

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.3. XPath Approach



- Consider for instance document trees A and B
  - The corresponding XPath representations
    - Following ***root path*** and ***set-based*** models

A_{set} = { ‘a’[1]/’b’[1]/’c’[1], ‘a’[1]/’b’[1]/’d’[1], ‘a’[1]/’b’[1]/’d’[2] }
B_{set} = { ‘a’[1]/’b’[1]/’e’[1], ‘a’[1]/’b’[1]/’e’[2] }

↳ Compared using any set-based similarity measure

- Complexity: **O(*k* × *N*)** where *N* = number of distinct root XPaths in A and B, and *k* is a variable factor

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.3. Path-based Similarity

3.3.3. XPath Approach

- NOTE:
  - The multi-set and vector representations can also be utilized with the XPaths
    - Even though it was originally designed to improve the set-based model
  - XPaths detect more structural clues than traditional path-based methods
    - Taking into some sibling information in addition to hierarchical relations

    ↳ Increasing structural **precision**

    
  - Yet they might miss certain sibling similarities
    - XPaths only take into account the number of siblings having the same label

    ↳ Decreasing overall **recall**

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

- NOTE:
  - One can realize that complexity levels of tag, edge, and path-based approaches are **average linear** in the number of tags/edges/paths in the document tree
    - In comparison with TED methods which are of **average polynomial** complexity
  - Also, transforming the document tree into the tag/edge/path based representation(s) requires **average linear** O(*N*) time where N is the number of tags/edges/paths
  - Yet, tag/edge/path methods are inherently less accurate than TED methods
    - Which are considered the **optimal techniques** for structure comparison

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.4. Fast Fourier Transform

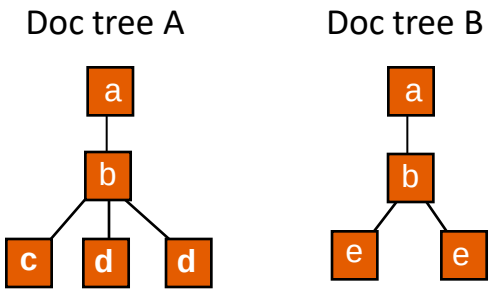
- An original semi-structured similarity approach [1], developed in the context of document clustering, consists of:
  - Step 1: Transforming the semi-structured document into a time series
  - Step 2: Transforming the time series into a frequency spectrum
  - Step 3: Comparing the resulting spectrums to evaluate structure similarity

[1] S. Flesca, G. Manco, E. Masciari, L. Pontieri and A. Pugliese. Detecting Structural Similarities Between XML Documents. *5th International ACM SIGMOD Workshop on The Web and Databases (WebDB),* (2002) pp. 55-60.

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.4. Fast Fourier Transform



- Step 1:
  - Producing the **skeleton** of the XML document
    - Sequence of **open** and **close tags**
      - Ordered following the order of traversal of the XML document
        - ⇨ Pre-order
  - Considering XML document trees A and B

Skeleton(A) = < <a>, , <c>, </c>, <d>, </d>, <d>, </d>, , >

Skeleton(B) = < <a>, , <e>, </e>, <e>, </e>, , >

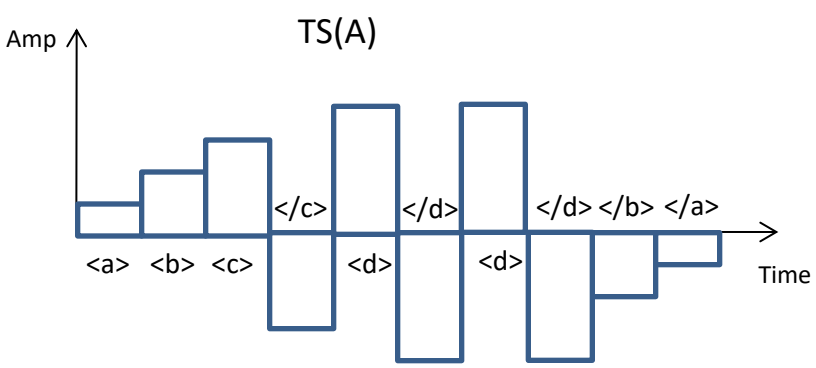
Each skeleton is transformed into a time series…

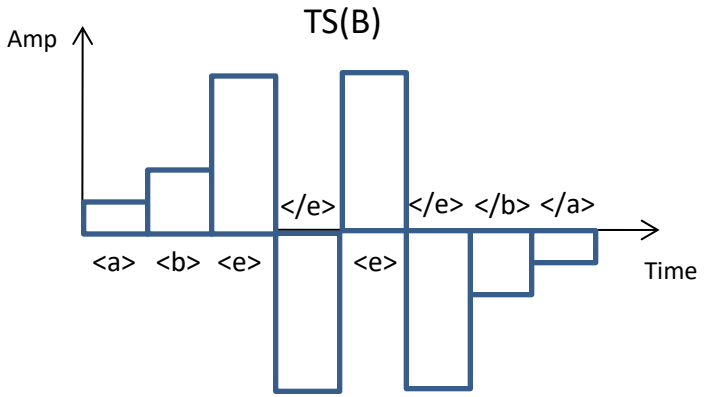
Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.4. Fast Fourier Transform

- Step 1:
  - To produce a time series representation
    - Each distinct opening tag is assigned a distinct amplitude value
      - Same amplitudes are assigned to the same tags in different skeletons
    - Each closing tag is assigned the negative amplitude of its opening tag
  - Considering: Skeleton(A) and Skeleton(B), we produce corresponding time series:





Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.4. Fast Fourier Transform

- Step 2:
  - Transforming time series into frequency spectrums using **Fast Fourier Transform**
    - Since comparing frequency spectrums is more robust than comparing time series
      - More resistant to **outliers** and **noise**

- Step 3:
  - Comparing frequency spectrums using any spectrum **correlation** measure
    - This can also be done using **vector-based** similarity measures
      - Since frequency spectrums can be represented as vectors where:
        - Frequency values represent dimensions
        - Frequency amplitudes represent vector weights

Ch. 6. Structural Similarity Approximation

3. Semi-structured Similarity Methods

3.4. Fast Fourier Transform

- Complexity: average logarithmic, e.g., **O(*N* × log(*N*))**
  - Where N represent the total number of opening and closing tags in the compared skeletons
    - Which simplifies to the total number of nodes in the compares XML trees

- However an experimental critique of the FFT approach shows that:
  - While it is **less complex** than TED-based methods
  - Yet the FFT approach runs **slower** than typical TED algorithms
  - Also, clustering experiments show that the FFT method always yields the **highest error rates** (largest number of mis-clustered documents)

    ⇨ Less accurate than certain edge-based and path-based variants

Ch. 6. Structural Similarity Approximation

Wrapping up

- Various approximations of TED-based similarity approaches exist
  - **Tag-based**, **Edge-based**, **Path-based**, and others (e.g., **FFT**)
- These can be used in a **filter-refinement** architecture with TED
  - Or as **standalone** measures in semi-structured data applications
    - In order to allow **faster** (even though **less accurate**) processing
- A **major advantage** of TED remains: producing an **edit script**
  - Central requirement in certain applications, e.g., data warehousing and integration
  - Provides a description of the similarity measures with tunable parameters (e.g., edit operations costs)
    - Which can be adapted following the user's needs

Ch. 6. Structural Similarity Approximation

Questions?

Ch. 5 - Structural Similarity

Main References

- Database Research Group, University of Salzburg, http://tree-edit-distance.dbresearch.uni-salzburg.at/
- Florian Wetzels, Heike Leitte, Christoph Garth: Accelerating Computation of Stable Merge Tree Edit Distances Using Parameterized Heuristics. IEEE Trans. Vis. Comput. Graph. 31(6): 3706-3718 (2025)
- Son Le Thanh, Tino Weinkauf: A Comparative Study of Different Edit Distance-Based Methods for Feature Tracking using Merge Trees on Time-Varying Scalar Fields. CoRR abs/2509.17974 (2025)
- Dayi Fan, Rubao Lee, Xiaodong Zhang: X-TED: Massive Parallelization of Tree Edit Distance. Proc. VLDB Endow. 17(7): 1683-1696 (2024)
- Zhongxi Fang, Jianming Huang, Xun Su, Hiroyuki Kasai: Wasserstein Graph Distance Based on L1-Approximated Tree Edit Distance between Weisfeiler-Lehman Subtrees. AAAI 2023: 7539-7549
- Masoud Seddighin, Saeed Seddighin 3+ε Approximation of Tree Edit Distance in Truly Subquadratic Time. ITCS 2022: 115:1-115:22
- S. Schwarz, M. Pawlik and N. Augsten. *A New Perspective on the Tree Edit Distance*. SISAP International Conference, 2017
- Joe Tekli, Gilbert Tekli, Richard Chbeir: Combining offline and on-the-fly disambiguation to perform semantic-aware XML querying. Comput. Sci. Inf. Syst. 20(1): 423-457 (2023)

Various other articles…